코드 설명 및 결과 분석 보고서

휴머노이드 예비단원 이주원

1. 서론

2. 코드 분석
    1. 라이브러리 및 파라미터 설정
    2. 환경 및 Q테이블 준비
    3. 행동 선택 함수
    4. 경험 수집과 Q 갱신
    5. 학습 평가
    6. 학습 성공률 곡선

3. 결과 분석
    1. 실험 조건
    2. 학습된 정책과 Q테이블
    3. 엡실론 감소 속도 변화에 따른 결과 분석
    4. 엡실론 최솟값 변화에 따른 결과 분석
    5. 학습률 변화에 따른 결과 분석
    6. 할인율 변화에 따른 결과 분석
    7. 미끄러짐 여부에 따른 결과 비교
    8. 미끄러짐 환경에서 학습된 정책 분석
    9. 에피소드 수 변화에 따른 결과 분석
    10. 전체적인 분석

4. MDP와 Q-learning 설명

1. 서론

이 과제는 강화학습의 기본 알고리즘인 Q러닝을 이용하여 프로즌레이크 환경을 학습시키는 것이다. 프로즌레이크는 4*4 크기의 얼음판에서 시작 칸부터 목표 칸까지 구멍에 빠지지 않고 이동하는 문제이다. 목표에 도착했을 때만 보상 1을 받고 나머지 경우에는 보상이 0이다. 이번 과제에서는 is_slippery=False로 설정하여 선택한 방향으로 정확히 이동하는 환경에서 실험하였다.

이 코드는 크게 총 6개의 부분으로 나누어져 있다. 첫번째로 라이브러리와 파라미터를 설정하는 부분이다. 학습에 필요한 라이브러리를 import하고 에피소드 수, 학습률, 할인율, 엡실

론 감소 속도 등 실험에 쓰이는 값들을 정의한다. 두번째로 환경과 Q테이블을 준비하는 부분이다. 프로즌레이크 환경을 만들고 16*4 크기의 Q테이블을 0으로 초기화한다. 세번째로 행동 선택 함수이다. 엡실론 그리디 방식으로 탐험과 활용 중 하나를 골라 행동을 결정한다. 네번째로 경험 수집과 Q 갱신 부분이다. 정해진 에피소드 수만큼 환경과 상호작용하며 Q러닝 방식으로 Q테이블을 갱신한다. 다섯번째는 학습 평가 부분이다. 저장한 Q테이블을 불러와 탐험 없이 200 에피소드를 실행하여 성공률을 구한다. 여섯번째는 학습 성공률 곡선을 그리는 부분이다. 학습 중 성공률의 변화를 500 에피소드 단위의 평균으로 나타낸다.

*실행방법은 README 파일을 참고

2. 코드분석

2-1 라이브러리 및 파라미터 설정

gymnasium으로 프로즌레이크 환경을 불러오고, numpy로 Q테이블 계산을, matplotlib으로 그래프 출력을 한다.
7~13줄은 실험에 쓰이는 파라미터 정의이다. 코드 앞부분에 모아두어 사용자가 값을 편하게 바꿀 수 있게 했다.num_episodes = 5000은 학습할 에피소드 수이다. 에피소드는 시작 칸에서 출발해 목표나 구멍에 도착하거나 걸음 제한으로 중단될 때까지의 한 번의 시행이다. alpha는 학습률로, 한 번 갱신할 때 큐값을 목표값 쪽으로 10퍼센트만 옮긴다. gamma는 할인율이다. eps_decay 와 eps_min은 엡실론 감소 속도와 최솟값이다.is_slippery = False이면 선택한 방향으로 정확히 이동한다.
15~19줄에서 설정한 파라미터를 출력하여 어떤 조건으로 실험했는지 로그에 남긴다.

2-2. 환경 및 Q테이블 준비

gym.make로 4*4 얼음판 환경을 만든다. 상태는 16개, 행동은 4개이다. Q = np.zeros((16, 4))로 Q테이블을 전부 0으로 초기화한다. Q[s, a]는 상태 s에서 행동 a를 했을 때 기대되는 미래 보상의 합이다. returns는 에피소드마다 받은 총 보상(성공 1, 실패 0)을 저장하는 리스트이다.

2-3. 행동 선택 함수

choose_action은 엡실론 그리디 방식으로 행동을 고른다. rng.random()이 epsilon보다 작으면 4개 중 무작위로 행동을 고른다. 그렇지 않으면 Q[state]에서 가장 큰 값을 가진 행동을 고른다.이때 np.flatnonzero로 최댓값이 여러 개인 행동을 모두 찾고 그중 하나를 무작위로 고른다. 처음에는 Q가 전부 0이다. argmax만 쓰면 항상 왼쪽만 골라 같은 자리에 갇히기 때문에, 동점일 때 무작위로 고르는 처리가 꼭 필요하다.

2-4. 경험 수집과 Q 갱신

num_episodes만큼 에피소드를 반복하며 학습한다. env.reset으로 시작 칸에서 출발한다. 시드는 첫 에피소드에만 주어 전체 실행을 재현 가능하게 한다. epsilon = max(eps_min, eps_decay ** episode)로 탐험 비율을 점점 줄인다. 기본 설정(0.999, 0.05)에서는 처음에 1에 가까워 거의 무작위로 움직이고, 약 3000 에피소드 이후에는 0.05로 고정되어 대부분 학습한 행동을 따를것으로 보인다. while 루프 안에서 행동을 고르고 env.step으로 한 칸 이동

한 뒤 다음 상태 ns, 보상 r, 종료 여부를 받는다. target은 Q-learning의 목표값이다. 구멍이나 목표에 도착해 terminated가 되면 그 뒤가 없으므로 target = r이다. 아니라면 target = r + gamma * Q[ns].max()로, 지금 받은 보상에 다음 상태에서 가장 좋은 행동의 가치를 할인해 더한다. Q[state, action] += alpha * (target - Q[state, action])으로 현재 값과 목표값의 차이의 alpha만큼 Q를 갱신한다. truncated(100덜음 넘으면 강제종료하는거)일 때는 상태가 끝난 것이 아니므로 다음 상태 값을 그대로 사용한다. 이 점이 terminated와 구분된다. 에피소드가 끝나면 총 보상을 returns에 저장하고, 학습이 끝나면 np.save로 큐테이블을 q_table.npy 파일로 저장한다.

2-5. 학습 평가

저장한 큐테이블을 np.load로 다시 불러와 학습 과정과 분리해서 평가한다. 평가용 환경 eval_env와 평가용 난수 eval_rng를 따로 만들어 학습에 쓴 환경과 섞이지 않게 한다. 미끄러짐 설정은 학습과 같은 is_slippery 값을 사용한다. epsilon = 0.0으로 두어 탐험 없이 Q값이 가장 큰 행동만 고른다.200 에피소드를 실행해 목표에 도착(r == 1)한 횟수를 세고 wins / 200으로 성공률을 출력한다.

2-6. 학습 성공률 곡선

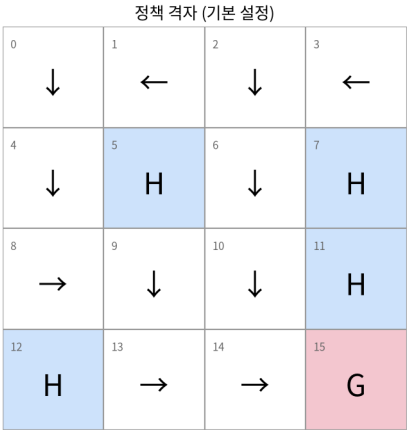
returns에 저장된 결과를 reshape(-1, 500)으로 500개씩 나누고, 구간마다 평균을 구한다. 각 값은 0 또는 1이므로 평균이 그 구간의 성공률이 될것이다. x축은 에피소드, y축은 0~1 사이의 성공률이다. 그래프 제목에 alpha, gamma, eps_decay 값을 넣어 어떤 조건의 결과인지 바로 알 수 있게 했다. 결과는 training_success_rate.png로 저장한다.
프로즌레이크는 에피소드의 총 보상이 성공 시 1, 실패 시 0이므로, 구간별 평균 보상 곡선과 성공률 곡선은 같은 그래프이다.

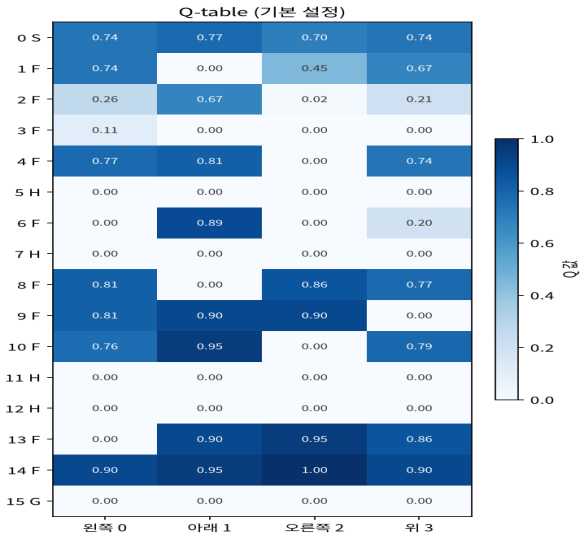
3. 결과분석

3-1. 실험 조건

변인 통제를 위해 시드를 0으로 고정하고, 한 번에 하나의 파라미터만 바꾸고 나머지는 기본값으로 고정하여 실험하였다. 평가는 시드 1000, 엡실론 0으로 200회 진행하였다. 미끄러짐이 없는 환경에서는 모든 설정에서 평가 결과가 200/200 성공, 평균 경로 길이 6.0으로 같았다. 6은 이 맵의 최단 경로 길이이므로 모든 설정에서 최적 경로를 찾은 것이다. 따라서 평가 결과로는 파라미터의 차이를 비교할 수 없었고, 학습 중 성공률과 시작 상태 Q값이 변하는 과정을 중심으로 분석하였다.

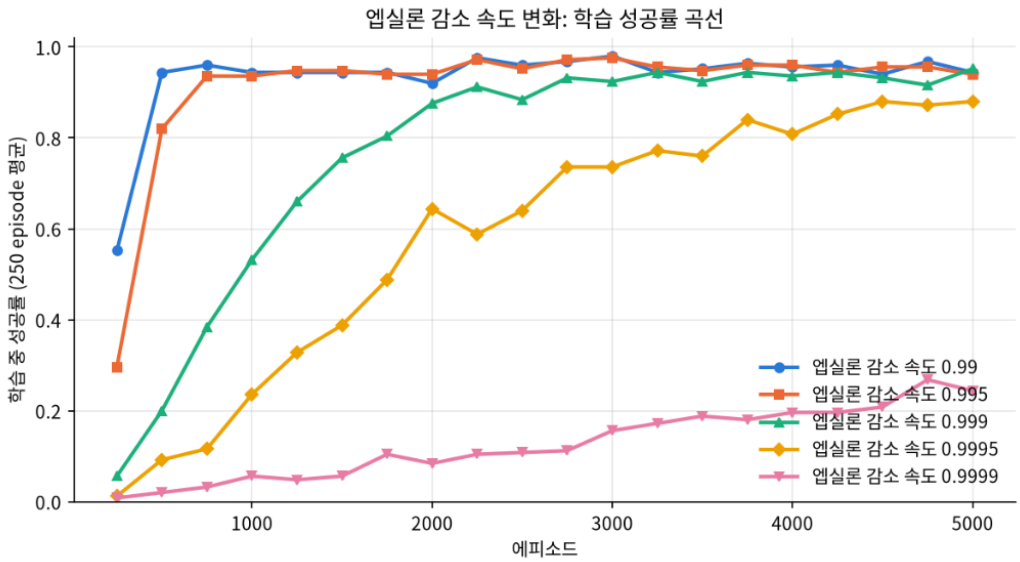
3-2. 학습된 정책과 Q테이블

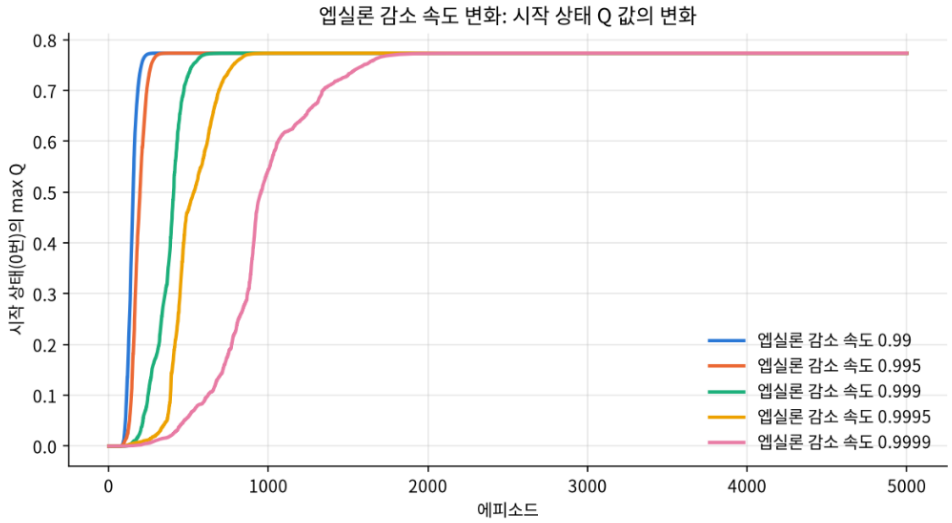




정책 격자는 각 칸에서 Q값이 가장 큰 행동을 화살표로 나타낸 것이다. 시작 칸에서 0 -> 4 -> 8 -> 9 -> 13 -> 14 -> 15 순서로 이동하며, 구멍을 모두 피해 6걸음 만에 목표에 도착한다. 이 경로 위의 Q값을 보면 0번 0.77, 4번 0.81, 8번 0.86, 9번 0.90, 13번 0.95, 14번 1.00이다. 이는 0.95^5, 0.95^4, ..., 0.95^0과 정확히 일치한다. 목표에서 받은 보상 1이 한 칸 뒤로 갈 때마다 할인율만큼 줄어들며 전파된 것이다.구멍과 목표 칸은 그 뒤가 없으므로 Q값이 모두 0이다. 또한 1, 2, 3번처럼 최적 경로에서 벗어난 칸은 Q값이 이론값보다 작은데, 학습 후반에 거의 방문하지 않아 갱신이 덜 된 것으로 보인다.

3-3. 엡실론 감소 속도 변화에 따른 결과 분석





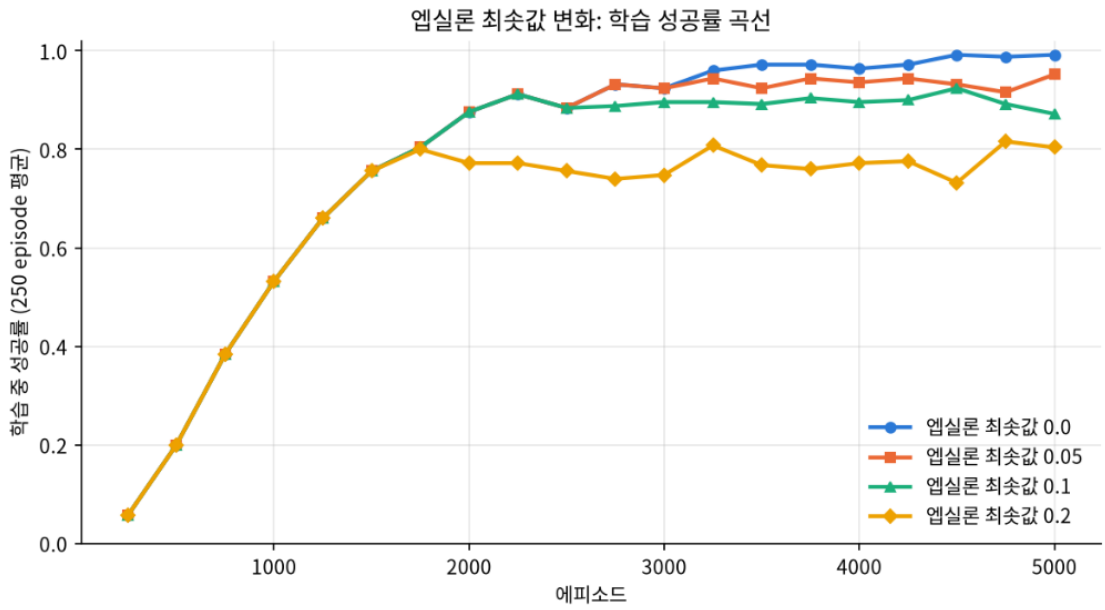
0.99, 0.995: 엡실론이 빠르게 줄어 500 에피소드 안에 학습 성공률 0.8에 도달하였다. 이 환경은 단순하고 미끄러지지 않아 적은 탐험으로도 경로를 빨리 찾을 수 있었다.

0.999: 학습 성공률이 천천히 올라가 1750 에피소드에서 0.8에 도달하였다.

0.9995, 0.9999: 탐험이 오래 유지되어 학습 성공률이 낮게 나왔다. 특히 0.9999는 5000 에피소드가 끝나도 엡실론이 약 0.6이어서 학습 성공률이 0.24에 그쳤다.하지만 시작 상태 큐

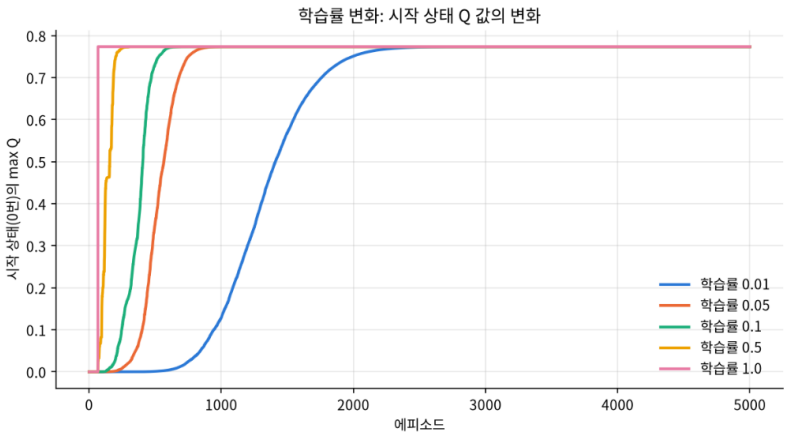
그래프를 보면 0.9999도 약 1800 에피소드에 0.774로 수렴하였고 평가에서도 200을 기록하였다. 학습 성공률이 낮은 이유는 Q테이블을 잘못 배워서가 아니라 학습 중에 무작위로 움직이는 비율이 높았기 때문으로 보인다. 이는 Q러닝이 실제 행동과 상관없이 다음 상태의 가장 좋은 행동을 기준으로 갱신하기 때문일 것이다. 무작위로 움직이는 동안에도 최적 정책의 가치를 배울 수 있다.

3-4. 엡실론 최솟값 변화에 따른 결과 분석



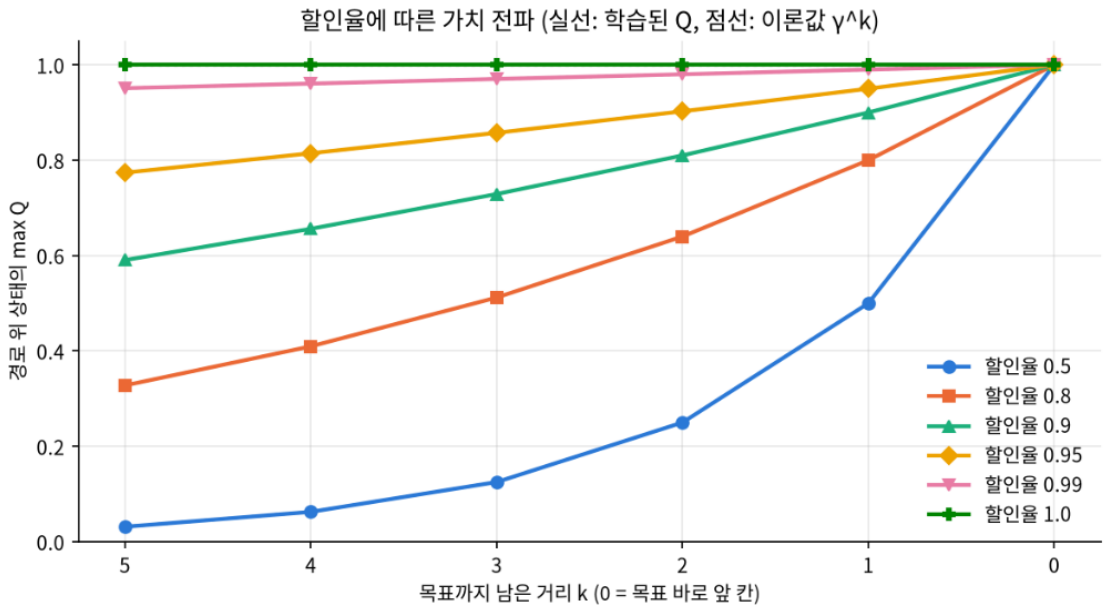
약 1750 에피소드까지는 모든 설정의 곡선이 같다. 엡실론이 아직 최솟값에 도달하지 않아 실제로 쓰이는 엡실론이 같기 때문이다.이후 최솟값이 0.0이면 학습 성공률이 0.99까지 올라갔지만, 0.2이면 약 0.77 부근에서 더 오르지 않았다. 6걸음 동안 매 걸음 20% 확률로 무작위 행동을 하면 구멍에 빠질 가능성이 커지기 때문이다.시작 상태 Q는 모든 설정에서 약 600 에피소드에 0.774로 같게 수렴하였다. 따라서 최솟값은 학습된 정책이 아니라 학습 중 성공률의 상한을 정하는 값이다.이번 환경은 단순해서 최솟값 0도 문제가 없었다. 하지만 복잡한 환경에서는 탐험을 완전히 끄면 더 좋은 경로를 찾지 못할 수 있다.

3-5. 학습률 변화에 따른 결과 분석



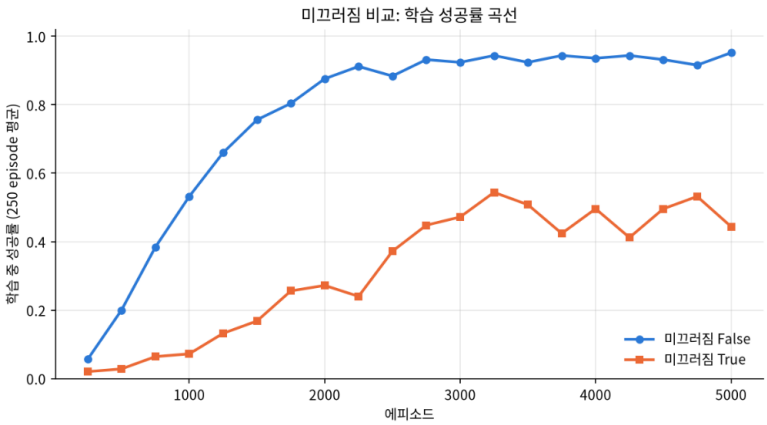
학습률이 클수록 시작 상태 Q가 빨리 수렴하였다. 1.0은 약 70 에피소드에서 곧바로 0.774가 되었고, 0.01은 약 2500 에피소드가 걸렸다.MNIST에서는 학습률 1.5 이상에서 학습이 망가졌지만 여기서는 1.0도 문제가 없었다. 미끄러지지 않는 환경에서는 같은 행동을 하면 항상 같은 결과가 나와서, 목표값을 한 번에 그대로 덮어써도 정확하기 때문이다.반면 학습 성공률 곡선은 학습률과 관계없이 거의 겹쳤다. 학습 중 행동은 엡실론에 의해 주로 결정되고, 경로 자체는 모든 설정에서 일찍 찾아졌기 때문이다.

3-6. 할인율 변화에 따른 결과 분석



경로 위 상태의 Q값은 목표에서 k칸 떨어져 있을 때 감마 k제곱이 되었고, 이론값과 정확히 일치하였다. 시작 상태 Q는 할인율 0.5에서 0.031(0.5^5), 0.8에서 0.328, 0.9에서 0.590, 0.99에서 0.951, 1.0에서 1.0이었다.할인율이 작을수록 먼 미래의 보상이 빠르게 작아진다. 0.5에서는 시작 칸의 가치가 0.03밖에 되지 않지만, 행동 간의 크기 비교는 유지되어 같은 최적 경로를 찾았다.할인율 1.0에서는 목표에 도착하는 모든 경로의 가치가 1로 같아진다. 그래서 이론적으로는 최단 경로를 더 선호할 이유가 없다. 이번 실험에서 경로 길이가 6.0으로 나온 것은 학습 과정에서 최단 경로가 먼저 강화되었기 때문으로 보인다. 할인율을 1보다 약간 작게 두는 것이 최단 경로를 찾는 데 더 안전하다.

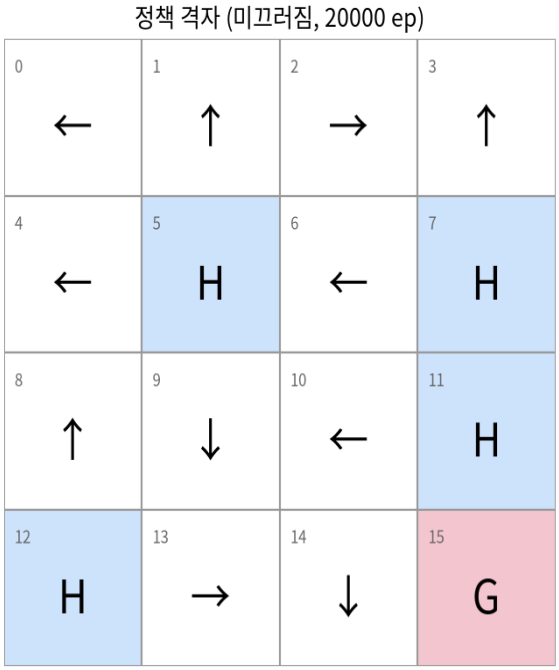
3-7. 미끄러짐 여부에 따른 결과 비교

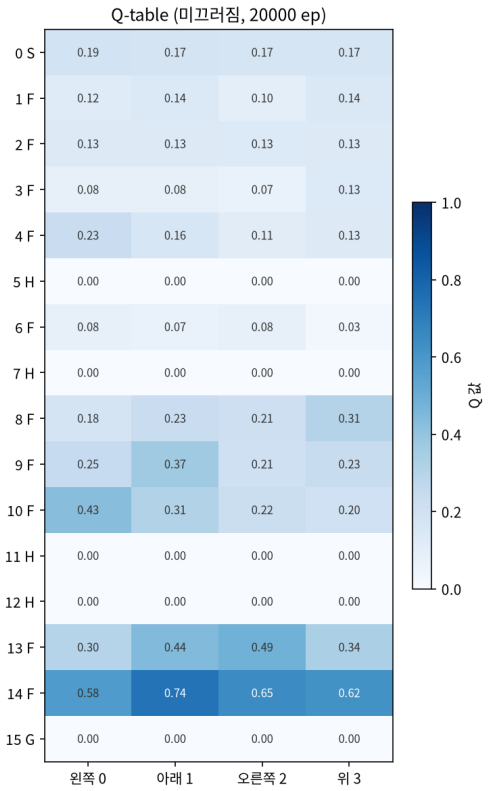




is_slippery=True이면 선택한 방향으로 갈 확률이 1/3이고, 양옆 방향으로 미끄러질 확률이 각각 1/3이다. 따라서 같은 상태에서 같은 행동을 해도 결과가 매번 달라진다. 학습 성공률 곡선을 보면 미끄러짐이 없을 때는 0.95까지 올라갔지만, 미끄러짐이 있을 때는 약 0.5에서 더 오르지 않았다. 로그를 보면 20000 에피소드까지 학습해도 학습 성공률은 0.38~0.59 사이를 오르내렸다. 시작 상태 Q값도 미끄러짐이 없을 때는 0.774로 깔끔하게 수렴하였다. 반면 미끄러짐이 있을 때는 약 0.17 근처에서 0.13~0.25 사이를 계속 흔들렸다. 같은 행동이라도 어떤 때는 목표에 가고 어떤 때는 구멍에 빠진다. 그래서 갱신할 때마다 목표값이 달라지고, 학습률 0.1로 계속 갱신하는 동안 Q값이 기대값 주변을 오르내린다. 시작 상태 Q가 0.774에서 약 0.18로 크게 낮아진 것은 목표에 도착할 확률 자체가 낮고 도착하기까지 걸음수도 많아 할인이 크게 적용되기 때문이다.

3-8. 미끄러짐 환경에서 학습된 정책 분석





미끄러짐이 없을 때의 정책과 다르게 미끄러짐이 있을 때의 정책은 목표와 전혀 맞지 않는 방향으로 가는것 처럼 보인다. 그러나 이는 발생할 수 밖에 없는 문제로 보인다.

어떤 방향을 고르면 그 방향과 양옆 방향으로는 갈 수 있지만, 정반대 방향으로는 절대 가지 않는다. 학습된 정책은 구멍이 있는 방향이 선택한 방향의 정반대가 되도록 행동을 고른다. 이를 뒷받침하는 예를 들어보면 4F에서는 구멍과 정반대 방향으로 가는 비율이 가장 높은 것으로 보이고 두번째로 높은 q값도 미끄러짐이 없을 때 q값이 많던 방향으로 가는 큐값이 높다. 단, 6F와 같은 경우는 예외로 화면에 왼쪽 화살표로 나와있어도 q값이 의미 있는 숫자는 아니고, 결국 6F로 최대한 지나가지 않는 방향으로 학습된 것 같다.

이러한 판단은 캐릭터가 몸에 붙이고 있다가 원하는 방향으로 미끄러질 때까지 기다리는 행동을 학습한 것으로 보인다. 평균 걸음걸이가 6걸음에서 38걸음으로 많은 수의 차이가 보인 것이 이 이유일 때문인 것으로 보인다. 빨리 가는 것이 아닌 구멍에 빠지지 않는 방향으로 학습된 결과인 것이다.

3-9. 에피소드 수 변화에 따른 결과 분석

greedy 경로는 68 에피소드에 이미 완성되었고, 시작 상태 Q는 약 600 에피소드에 수렴하였다. 따라서 미끄러짐이 없는 환경에서는 500 에피소드만으로도 평가 200/200을 기록하였다. 에피소드 수가 늘어나도 학습 성공률이 올라가는 것은 정책이 좋아져서가 아니라 엡실론이 줄어들었기 때문으로 보인다.

3-10. 전체적인 분석

미끄러짐이 없는 프로즌레이크는 단순한 환경이라 모든 설정에서 최적 경로를 찾았다. 파라미터는 결과보다 학습 속도와 학습 중의 행동에 영향을 주었다.학습률은 Q값이 수렴하는 속도를, 할인율은 Q값의 크기를, 엡실론 관련 값은 학습 중 성공률을 주로 결정하였다.또한 학습 중 성공률과 실제 학습된 정책의 품질은 다를 수 있다는 것을 확인하였다. 그래서 학습과 평가를 분리하여 엡실론 0으로 따로 평가하는 과정이 필요하다.미끄러짐이 있는 환경에서는 같은 설정으로도 성능이 크게 떨어졌다. 환경이 확률적일수록 더 많은 학습이 필요하다는 것을 알 수 있었다.

4. MDP와 Q-learning 설명

4-1. MDP 요소

프로즌레이크는 다섯가지의 요소를 갖는 MDP이다.
먼저 상태 s를 갖는다. 구멍 4개와 골지점 1개를 갖고, 0부터15까지 총 16개이다.
행동 a 는 왼쪽, 아래, 오른쪽, 위 총 4개를 갖는다.
전이확률 p를 갖는다. 미끄러짐이 없는 상태는 확률 1을 갖는다.
보상 r, 목표에 도착하면 1 아니면 0이다.
100걸음을 걸으면 상태와 관련 없이 시간제한 때문에 에피소드가 중단된다.

4-2 Return

리턴은 어떤 시점부터 받는 보상을 할인하여 모두 더한 값이다.$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$최단 경로 6걸음으로 목표에 가면 보상은 0, 0, 0, 0, 0, 1 순서로 받는다. 따라서 시작 칸에서의 리턴은$G_0 = 0 + 0.95\times0 + \cdots + 0.95^5 \times 1 = 0.774$ 이다.실험에서 학습된 시작 상태의 Q값도 0.7738로, 이 값과 일치하였다.

4-3 q업데이트 수치 예제

Q 업데이트 식은 다음과 같다.Q(s,a) <- Q(s,a) + α [r + γ max Q(s',a') − Q(s,a)]α = 0.1, γ = 0.95이고 Q는 모두 0에서 시작한다.
예제 1: 14번 칸에서 오른쪽으로 가서 목표에 처음 도착한 경우이다. 종료 상태이므로 목표값은 r = 1이다. Q(14, 오른쪽) = 0 + 0.1 × (1 − 0) = 0.1
예제 2: 그 뒤 10번 칸에서 아래로 가서 14번 칸에 도착한 경우이다. 보상은 0이고, 14번 칸의 최댓값은 0.1이다. 목표값 = 0 + 0.95 × 0.1 = 0.095Q(10, 아래) = 0 + 0.1 × (0.095 − 0) = 0.0095
예제 3: 14번 칸에서 오른쪽으로 다시 목표에 도착한 경우이다. Q(14, 오른쪽) = 0.1 + 0.1 × (1 − 0.1) = 0.19이렇게 같은 행동이 반복될수록 Q(14, 오른쪽)은 1 − 0.9^n으로 1에 가까워진다. 그 앞 칸들은 0.95, 0.95^2 … 로 수렴하며, 목표의 보상이 한 칸씩 뒤로 전파된다.